```
import zipfile

zip_path = "/content/sample_data/archive.zip"

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall("/content/dataset")
```

```
import os
os.listdir("/content/dataset")
```

```
['no', 'yes', 'brain_tumor_dataset']
```

```
import shutil
shutil.rmtree("/content/dataset/brain_tumor_dataset")
```

```
from torchvision import transforms

transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor(),
    transforms.Normalize([0.5,0.5,0.5], [0.5,0.5,0.5])
])
```

```
from torchvision.datasets import ImageFolder

dataset = ImageFolder(
    root="/content/dataset",
    transform=transform
)

print("Total images:", len(dataset))
print("Classes:", dataset.classes)
```

```
Total images: 253
Classes: ['no', 'yes']
```

```
from torch.utils.data import DataLoader, random_split

train_size = int(0.8 * len(dataset))
test_size = len(dataset) - train_size

train_data, test_data = random_split(dataset, [train_size, test_size])

train_loader = DataLoader(train_data, batch_size=16, shuffle=True)
test_loader = DataLoader(test_data, batch_size=16)
```

```
import torch
import torch.nn as nn
from torchvision import models

model = models.resnet18(pretrained=True)
model.fc = nn.Linear(model.fc.in_features, 2)
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print("Using device:", device)
```

```
Using device: cpu
```

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

```
num_epochs = 5

for epoch in range(num_epochs):
    model.train()
    running_loss = 0
    correct = 0
    total = 0
```

```
    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()

        outputs = model(images)
        loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, predicted = torch.max(outputs, 1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    acc = 100 * correct / total
    print(f"Epoch [{epoch+1}/{num_epochs}] Loss: {running_loss:.4f} Accuracy: {acc:.2f}%")
```

```
Epoch [1/5] Loss: 10.1306 Accuracy: 73.27%
Epoch [2/5] Loss: 4.4540 Accuracy: 84.65%
Epoch [3/5] Loss: 2.2518 Accuracy: 94.06%
Epoch [4/5] Loss: 2.1975 Accuracy: 92.57%
Epoch [5/5] Loss: 1.4250 Accuracy: 95.05%
```

```
torch.save(model.state_dict(), "/content/brain_tumor_resnet18.pth")
print("✅ Model saved")
```

```
✅ Model saved
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, confusion_matrix
import numpy as np

model.eval()

y_true = []
y_pred = []

with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs, 1)

        y_true.extend(labels.numpy())
        y_pred.extend(predicted.cpu().numpy())

print("Accuracy:", accuracy_score(y_true, y_pred))
print("Precision:", precision_score(y_true, y_pred))
print("Recall:", recall_score(y_true, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_true, y_pred))
```

```
Accuracy: 0.8627450980392157
Precision: 0.8421052631578947
Recall: 0.9696969696969697
Confusion Matrix:
 [[12  6]
 [ 1 32]]
```

```
import matplotlib.pyplot as plt

images, labels = next(iter(test_loader))
images = images.to(device)

outputs = model(images)
_, preds = torch.max(outputs, 1)

images = images.cpu()

plt.figure(figsize=(10,6))
for i in range(6):
    plt.subplot(2,3,i+1)
    plt.imshow(images[i].permute(1,2,0))
    plt.title("Tumor" if preds[i]==1 else "No Tumor")
```

```
    plt.axis("off")
plt.show()
```

```
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for inte
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for inte
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for inte
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for inte
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for inte
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for inte
```